

مهمة تحليل شاملة — تضارب قواعد المنافع والمجموعات والأوعية المالية

أنت تعمل داخل مستودع منظومة WAAD TPA المبنية باستخدام:

- Spring Boot / Java
- PostgreSQL
- Hibernate / JPA
- Flyway
- Next.js / React

نطاق المهمة

حلل بصورة كاملة الهيكل الحالية المتعلقة بـ:

- BenefitPolicy
- BenefitPolicyRule
- BenefitGroup
- BenefitLimitBucket
- BenefitRuleBucket

- محرك التغطية.
- محرك الاستهلاك المالي.
- محرك المطالبات.
- محرك الموافقات المسبقة.
- محاكاة التغطية.

• واجهة إعداد قواعد الوثائق، خصوصاً: BenefitPolicyRulesTab.jsx

المشكلة المراد دراستها هي احتمال وجود التصنيف الطبي نفسه في الوثيقة بصورة مزدوجة:

1. كقاعدة منفردة لها نسبة تغطية أو سقف مالي.
2. كعضو داخل مجموعة منافع مرتبطة بوعاء مالي مشترك.

مثال محتمل:

- التصنيف: الأجهزة والمعدات الطبية CAT-DME
- يظهر كتصنيف فردي بسقف 1500.
- ويظهر مرة أخرى داخل مجموعة منافع بسقف مشترك 1500.

المطلوب هو التحليل واستخراج الأدلة فقط.

لا تعدل الكود. لا تعدل قاعدة البيانات. لا تضيف Migration. لا تحذف أي ملف. لا تنفذ Refactoring. لا تصلح المشكلة الآن.

أولاً: الأسئلة الأساسية المطلوب حسمها

يجب أن يجيب التقرير بدقة عن الأسئلة التالية:

1. هل يستطيع التصنيف الطبي نفسه أن يوجد أكثر من مرة داخل الوثيقة نفسها؟
2. هل يستطيع أن يوجد مرة كقاعدة مستقلة ومرة داخل مجموعة؟
3. هل المجموعة تحتوي قواعد المنافع نفسها أم تحتوي التصنيفات مباشرة؟
4. هل `BenefitRuleBucket` يسمح بربط القاعدة بأكثر من وعاء؟
5. هل السقف المالي مخزن داخل:
6. `BenefitPolicyRule`
7. `BenefitLimitBucket`
8. الاثنين معاً؟
9. ما المصدر الحقيقي المعتمد للسقف أثناء معالجة المطالبة؟
10. ما المصدر الحقيقي لنسبة التغطية؟
11. هل المجموعة كيان مالي فعلي أم مجرد كيان تنظيمي؟
12. هل المجموعة مرتبطة بوعاء واحد أم قد ترتبط بأكثر من وعاء؟
13. هل الوعاء قد يكون مشتركاً بين أكثر من مجموعة؟
14. هل القاعدة المستقلة يجب أن ترتبط بوعاء؟
15. هل توجد قواعد لا ترتبط بأي وعاء؟
16. هل توجد قواعد مرتبطة بأكثر من وعاء؟
17. ماذا يحدث عند وجود `amountLimit` داخل القاعدة ووعاء مرتبط بها في الوقت نفسه؟
18. هل يخصم محرك المطالبات من:
 - سقف القاعدة؟
 - الوعاء؟
 - الاثنين؟
 - أحدهما حسب الأولوية؟
19. هل تختلف النتيجة بين:
 - المطالبات.
 - الموافقات المسبقة.
 - محاكاة التغطية.
 - عرض الوثيقة في الواجهة؟
20. هل توجد قيود في قاعدة البيانات تمنع التكرار؟
21. هل توجد `Validation` في طبقة `Service` تمنع التكرار؟
22. هل واجهة المستخدم تمنع التكرار فقط بصرياً أم لا تمنعه مطلقاً؟
23. هل قائمة إضافة التصنيف تستبعد التصنيفات الموجودة مسبقاً داخل مجموعة؟
24. هل يمكن إرسال طلب `API` يضيف تصنيفاً مكرراً حتى لو منعه الواجهة؟
25. هل يوجد `Race Condition` يسمح بطلبين متزامنين بإضافة التصنيف نفسه؟
26. هل `OP` و `IP` يعتبران نطاقين مختلفين يسمح فيهما بوجود التصنيف مرتين؟
27. هل تختلف القواعد حسب:
 - الوثيقة.
 - إصدار الوثيقة.
 - جهة العمل.
 - مجموعة المستفيدين.
 - الفترة الزمنية.
28. ما أثر حذف المجموعة على القواعد والأوعية والاستهلاك؟
29. ما أثر إزالة قاعدة من المجموعة؟

30. ما أثر نقل تصنيف مستقل إلى مجموعة؟
31. هل يتم الحفاظ على الاستهلاك المالي السابق عند النقل؟
32. هل يوجد Audit Log لهذه العمليات؟
33. هل البيانات الحالية تحتوي حالات تضارب فعلية؟

ثانياً: تحليل نموذج البيانات

افحص جميع الكيانات ذات العلاقة، بما فيها:

- .Entities
- .Embeddables
- .Join tables
- .Repositories
- .DTOs
- .Mappers
- .Enums
- .Flyway migrations
- .Database constraints
- .Indexes
- .Cascade rules
- .Orphan removal
- .Soft deletion
- .Versioning
- .Audit fields

أنشئ مخططاً نمياً واضحاً للعلاقات، مثلاً:

```
BenefitPolicy
├── BenefitPolicyRule
│   ├── BenefitRuleBucket
│   └── BenefitLimitBucket
└── BenefitGroup
    └── ...
```

لكن يجب أن يعكس المخطط العلاقات الحقيقية في المشروع، لا العلاقات المفترضة.

لكل علاقة وُضح:

- نوع العلاقة.
- اتجاهها.
- المفتاح الأجنبي.
- هل هي إلزامية أم اختيارية؟
- هل تسمح بالتعدد؟
- سياسة الحذف.
- القيود الفريدة.
- أين تُستخدم داخل الكود؟

ثالثاً: تحليل الحقول المالية

أنشئ جدولاً شاملاً لكل حقل مالي في هذه الكيانات:

الكيان	الحقل	النوع	المعنى	أين يُكتب؟	أين يُقرأ؟	هل هو مصدر حقيقة؟
--------	-------	-------	--------	------------	------------	-------------------

يشمل ذلك، حسب الموجود فعلياً:

- amountLimit
- coveragePercentage
- copay
- deductible
- remainingAmount
- consumedAmount
- periodLimit
- timesLimit
- familyLimit
- memberLimit
- bucketLimit
- أي حقل مالي آخر

حدد بصورة صريحة:

- الحقول القديمة.
- الحقول المستخدمة فعلياً.
- الحقول المعروضة فقط.
- الحقول التي تؤثر على الحساب.
- الحقول التي تتعارض مع بعضها.
- الحقول الموجودة في قاعدة البيانات لكن لا يستخدمها المحرك.
- الحقول التي يكتبها Frontend ولا يقرأها Backend.
- الحقول التي يقرأها Backend ولا تظهر للمستخدم.

رابعاً: تتبع محرك التغطية

ابدأ من نقطة إنشاء أو مراجعة المطالبة، ثم تتبع المسار الكامل حتى القرار المالي النهائي.

يجب توثيق المسار الفعلي بالشكل التالي:

Controller

- Application Service
- Coverage Resolver
- Rule lookup
- Group lookup
- Bucket lookup

- Consumption calculation
- Financial snapshot
- Ledger write

لكل خطوة اذكر:

- اسم الملف.
- اسم الكلاس.
- اسم الدالة.
- رقم السطر التقريبي.
- المدخلات.
- المخرجات.
- القرار الذي تتخذه.
- الحقول التي تعتمد عليها.
- ماذا تفعل عند وجود أكثر من قاعدة أو وعاء؟

تحقق من جميع المسارات التالية بصورة منفصلة:

1. Claim creation
2. Claim approval
3. Claim financial calculation
4. Pre-Authorization creation
5. Pre-Authorization approval
6. Coverage simulation
7. Eligibility check
8. Provider portal preview
9. Recalculation أو correction
10. Settlement calculation، إذا كان يعتمد على هذه القيم.

خامساً: إثبات أو نفي الخصم المزدوج

لا تفترض وجود خصم مزدوج.

أثبت أو انفي ذلك من الكود والاختبارات.

اختبر نظرياً أو عملياً، من دون تعديل الكود، السيناريو التالي:

Policy: P1
Category: CAT-DME

Individual rule:
coverage = 80%
amount limit = 1500

Shared group:
G-CAT026
bucket limit = 1500

• موجود داخل المجموعة أيضاً CAT-DME

Claim amount = 1000

استخرج النتيجة الفعلية المتوقعة من المحرك:

- ما القاعدة المختارة؟
- ما الوعاء المختار؟
- هل تُقرأ القاعدة الفردية؟
- هل تُقرأ عضوية المجموعة؟
- هل يخصم من وعاء واحد أم اثنين؟
- ماذا يحدث للرصيد؟
- ماذا يظهر في Financial Snapshot؟
- ماذا يظهر في Ledger؟
- ماذا يحدث عند تكرار المطالبة؟

صنّف النتيجة إلى واحدة من الحالات:

- DOUBLE_DEDUCTION_CONFIRMED •
- SINGLE_BUCKET_SELECTED •
- INDIVIDUAL_LIMIT_SELECTED •
- SHARED_BUCKET_SELECTED •
- AMBIGUOUS_SELECTION •
- NON_DETERMINISTIC •
- VALIDATION_PREVENTS_STATE •
- STATE_EXISTS_BUT_ENGINE_IGNORES_ONE_SIDE •
- غير ذلك مع التفسير

سادساً: تحليل واجهة المستخدم

افحص `BenefitPolicyRulesTab.jsx` وكل المكونات والخدمات المرتبطة به.

تحقق من:

- مصدر `benefitRows`.
- مصدر `groupRows`.
- سبب استخدام:

```
return [...benefitRows, ...groupRows];
```

- هل التصنيف داخل المجموعة يظهر أيضاً ضمن `benefitRows`؟
- هل يوجد تمييز بين:
- قاعدة مستقلة.
- قاعدة مرتبطة بمجموعة.
- مجموعة.
- وعاء.

- هل الواجهة تستنتج حالة الربط أم تستلمها من Backend؟
- هل يمكن أن تعرض شارة «بلا سقف أو مجموعة» مع وجود سقف فعلي؟
- هل أسماء الحقول في الواجهة تعكس معناها الحقيقي؟
- هل هناك خلط بين «المجموعة» و«الوعاء»؟
- هل التكرار بصري فقط أم يعكس تكراراً فعلياً في البيانات؟
- هل Expand/Collapse موجود حالياً؟
- هل توجد عملية نقل بين مستقل ومجموعة؟
- هل الحذف من المجموعة يحذف القاعدة أم الارتباط فقط؟
- هل تحديث السقف يحدث في القاعدة أم الوعاء؟

أنشئ جدولاً:

عملية الواجهة API المستخدم الكيان المتأثر النتيجة الفعلية الملاحظة

يشمل:

- إضافة تصنيف.
- تعديل تصنيف.
- حذف تصنيف.
- إنشاء مجموعة.
- تعديل مجموعة.
- إضافة تصنيف إلى مجموعة.
- إزالة تصنيف من مجموعة.
- تعديل السقف.
- تعديل نسبة التغطية.
- حذف مجموعة.

سابعاً: تحليل واجهات API

استخرج جميع Endpoints المرتبطة بالوئات والقواعد والمجموعات والأوعية.

لكل Endpoint اذكر:

Method Endpoint Controller Service العملية Validation المخاطر

تحقق مما إذا كان يمكن عبر API:

- إنشاء قاعدة مكررة.
- إنشاء عضوية مكررة.
- ربط قاعدة بأكثر من وعاء.
- إرسال `amountLimit` و `bucketId` معاً.
- تغيير `groupId` من دون إزالة الربط القديم.
- حذف وعاء مستخدم.
- حذف مجموعة تحتوي قواعد.
- إنشاء مجموعة بتصنيف واحد.
- إنشاء مجموعة فارغة.

- إضافة التصنيف نفسه إلى مجموعتين.
- تجاوز قيود الواجهة.

ثامناً: تحليل قاعدة البيانات والترحيلات

افحص جميع Flyway migrations التي أنشأت أو عدّلت:

- Benefit policies
- Benefit rules
- Benefit groups
- Benefit buckets
- Rule-bucket links
- Consumption tables
- Financial ledger
- Claim snapshots

استخرج التسلسل التاريخي للبنية:

Migration	التغيير	سبب محتمل	هل مازال مستخدماً؟	هل خُلف ازدواجية؟
-----------	---------	-----------	--------------------	-------------------

تحقق من وجود:

- عمود أضيف ثم أصبح غير مستخدم.
- جدول وسيط قديم.
- أكثر من طريقة لتمثيل العلاقة نفسها.
- قيود أزيلت.
- حقول `amount_limit` قديمة بقيت بعد إدخال الأوعية.
- جداول تحمل أسماء متشابهة.
- Migrations تصحيحية مراكمة.
- بيانات Seed قد تنشئ ازدواجية.

لا تعدّل الترحيلات: اكنف بتحليلها.

تاسعاً: فحص البيانات الفعلية

نقذ استعلامات قراءة فقط لاكتشاف الحالات التالية:

1. تصنيف موجود أكثر من مرة في الوثيقة والسياق نفسه.
2. تصنيف موجود كقاعدة مستقلة وعضو مجموعة.
3. قاعدة مرتبطة بأكثر من وعاء.
4. قاعدة بلا وعاء.
5. مجموعة بلا وعاء.
6. مجموعة بلا أعضاء.
7. مجموعة بعضو واحد.
8. وعاء بلا قواعد.
9. وعاء مرتبط بأكثر من مجموعة.

10. سقف داخل القاعدة مع وجود وعاء.
11. اختلاف السقف الفردي عن سقف الوعاء.
12. استهلاك مرتبط بقاعدة غير مرتبطة بوعاء.
13. استهلاك يتجاوز السقف.
14. تصنيفات متكررة بين مجموعات مختلفة.
15. اختلاف OP و IP غير المبرر.
16. قواعد محذوفة منطقياً وما زالت مرتبطة.
17. مجموعات محذوفة وما زالت مستخدمة.
18. Claims أو Snapshots تشير إلى بنية قديمة.

أرفق استعلام SQL المستخدم لكل حالة.

أخرج النتائج في جدول:

نوع التعارض	العدد	أمثلة	الخطورة
-------------	-------	-------	---------

لا تعرض بيانات شخصية أو طبية حساسة.

عاشراً: اختبار التزامن

حلّل ما إذا كان طلبان متزامنان يستطيعان:

- إضافة القاعدة نفسها.
- إضافة التصنيف نفسه إلى مجموعتين.
- نقل القاعدة أثناء إنشاء مطالبة.
- تعديل السقف أثناء الخصم.
- إنشاء وعاءين لنفس القاعدة.
- حذف المجموعة أثناء استخدامها.

افحص:

- Unique constraints
- Transaction boundaries
- Isolation level
- Pessimistic locks
- Optimistic locking
- @Version
- Service validation
- Database enforcement

وضح لكل سيناريو:

السيناريو	الحماية الحالية	كافية؟	النتيجة المحتملة
-----------	-----------------	--------	------------------

الحادي عشر: تحليل المصطلحات

حدّد المعنى الفعلي لكل مصطلح في النظام:

- Rule.
- Group.
- Bucket.
- Limit.
- Coverage.
- Membership.
- Consumption.
- Ledger.
- Snapshot.
- Individual benefit.
- Shared benefit.

حدد إن كانت الأسماء الحالية:

- صحيحة.
- مضللة.
- قديمة.
- تستخدم بمعانٍ مختلفة بين Frontend و Backend.

أنشئ معجماً مختصراً:

المصطلح الحالي	المعنى الفعلي	المشكلة	الاسم الأنسب
----------------	---------------	---------	--------------

لا تنفذ إعادة تسمية، فقط اقترحها في التقرير.

الثاني عشر: مقارنة النماذج المعمارية الممكنة

من دون اختيار حل نهائي، قارن النماذج التالية:

النموذج A

السقف داخل BenefitPolicyRule ، والمجموعة اختيارية.

النموذج B

السقف داخل BenefitLimitBucket فقط، وكل قاعدة ترتبط بوعاء واحد.

النموذج C

القاعدة قد تكون مستقلة بلاء أو داخل مجموعة بوعاء.

النموذج D

المجموعة هي الكيان المالي نفسه ولا يوجد وعاء منفصل.

النموذج الحالي الفعلي

صفه كما هو موجود في الكود.

قارن بينها حسب:

المعيار	A	B	C	D	الحالي
منع التكرار					
وضوح مصدر السقف					
دعم السقف المشترك					
سهولة الاستعلام					
سلامة الخصم					
قابلية التوسع					
سهولة الواجهة					
سهولة الترحيل					

لا تصدر قراراً نهائياً من دون ربط المقارنة بحالة المشروع الفعلية.

الثالث عشر: التقرير المطلوب

أنشئ الملف:

docs/audits/BENEFIT_RULE_GROUP_BUCKET_FULL_ANALYSIS.md

ويجب أن يحتوي الأقسام التالية:

1. الملخص التنفيذي

- وصف المشكلة.
- هل هي مشكلة عرض فقط أم مشكلة بيانات ومحرك؟
- مستوى الخطورة.
- أثرها المالي.
- أثرها التشغيلي.
- أثرها على قابلية التوسع.

2. الهيكلية الحالية

- الكيانات.
- العلاقات.
- الحقول المالية.
- القيود.
- مخطط العلاقات.

3. مسارات التنفيذ

- المطالبات.
- الموافقات.
- المحاكاة.
- الاستهلاك.
- التسويات.

4. نتائج فحص البيانات

- أعداد الحالات المتضاربة.
- أمثلة غير حساسة.
- استعلامات SQL.

5. نتائج فحص الواجهة

- سبب التكرار البصري.
- هل الواجهة تكشف المشكلة أم تسببها؟
- عمليات الإضافة والتعديل والحذف.

6. نتائج فحص Backend

- Validation.
- APIs.
- التزامن.
- مصدر الحقيقة.

7. نتيجة اختبار الخصم

- هل يوجد خصم مزدوج؟
- الدليل.
- السيناريو.
- النتيجة.

8. الديون التقنية

- الكود القديم.
- الحقول القديمة.
- الجداول أو العلاقات المكررة.
- الترحيلات المتراكمة.
- الملفات الضخمة.
- المسؤوليات المختلطة.

9. الخيارات المعمارية

لكل خيار:

- الوصف.
- المزايا.
- العيوب.
- أثره على البيانات.
- أثره على Frontend.
- أثره على محرك المطالبات.
- أثره على الترحيلات.
- المخاطر.

10. التوصية الأولية

قدم توصية أولية فقط، مع توضيح:

- لماذا هي الأنسب؟
- ما المعلومات التي قد تغير القرار؟
- ما القرارات التجارية التي تحتاج تأكيداً من مالك النظام؟

11. الأسئلة التي تحتاج قراراً من مالك النظام

يجب أن تشمل على الأقل:

1. هل كل تصنيف يجب أن يرتبط بسقف؟
2. هل يسمح بتصنيف بلا سقف؟
3. هل السقف الفردي مستقل عن السقف المشترك؟
4. هل يسمح للتصنيف نفسه بقواعد مختلفة في OP و IP؟
5. هل أعضاء المجموعة قد يحملون نسب تغطية مختلفة؟
6. هل يمكن للمجموعة أن تضم تصنيفاً واحداً؟
7. ماذا يحدث للاستهلاك عند نقل التصنيف؟
8. هل السقف على الفرد أم العائلة أم الوثيقة؟
9. هل المجموعة تنظم بصري أم وحدة مالية؟
10. ما سياسة حذف المجموعة؟

12. جدول الأدلة

النتيجة	الحالة	الدليل	الملف	السطر	الثقة
---------	--------	--------	-------	-------	-------

استخدم الحالات:

- CONFIRMED
- PARTIALLY_CONFIRMED
- NOT_FOUND
- FALSE_POSITIVE
- REQUIRES_BUSINESS_DECISION

الرابع عشر: ملفات مساندة مطلوبة

بالإضافة إلى التقرير الرئيسي، أنشئ:

1. docs/audits/BENEFIT_MODEL_ENTITY_MAP.md
2. docs/audits/BENEFIT_MODEL_API_MAP.md
3. docs/audits/BENEFIT_MODEL_DATABASE_FINDINGS.md
4. docs/audits/BENEFIT_MODEL_FRONTEND_FINDINGS.md
5. docs/audits/BENEFIT_MODEL_ENGINE_TRACE.md

الخامس عشر: صيغة الخلاصة النهائية

في نهاية التقرير اكتب خلاصة واضحة بهذا الشكل:

:الحكم النهائي

[مشكلة عرض فقط / مشكلة بيانات / مشكلة محرك / مشكلة مركبة]

هل الخصم المزدوج مثبت؟

[نعم / لا / محتمل / غير قابل للإثبات حالياً]

هل التصنيف يمكن أن يوجد مستقلاً وداخل مجموعة؟

[نعم / لا]

هل قاعدة البيانات تمنع ذلك؟

[نعم / لا / جزئياً]

يمنع ذلك؟ Backend هل

[نعم / لا / جزئياً]

يمنع ذلك؟ Frontend هل

[نعم / لا / جزئياً]

:مصدر السقف الفعلي

[...]

:مصدر نسبة التغطية

[...]

:أكبر خطر

[...]

:أفضل الخيارات المعمارية المرشحة

1. [...]

2. [...]

3. [...]

:القرارات التجارية المطلوبة

[...]

:الملفات الحرجة التي يجب تعديلها لاحقاً

[...]

السادس عشر: ضوابط جودة التقرير

- لا تستخدم عبارات عامة بلا دليل.
- لا تقل «قد يحدث» من دون تحديد مسار حدوثه.
- لا تعتبر التكرار البصري دليلاً تلقائياً على الخصم المزدوج.
- لا تفترض أن اسم الكيان يعكس وظيفته الحقيقية.
- لا تعتمد على Frontend وحده لتفسير نموذج البيانات.
- لا تعتمد على Entity وحده دون فحص Flyway.
- لا تعتمد على Service وحده دون فحص قيود PostgreSQL.
- لا تعتبر Validation البرمجي بديلاً عن قيد قاعدة البيانات.
- لا تعرض بيانات حساسة.
- لا تعدل أي ملف تنفيذي.
- لا تنشئ Migration.
- لا تنفذ Cleanup.
- لا تقدم حلاً نهائياً قبل إكمال التقرير.

ابدأ بتحليل الكيانات والترحيلات، ثم تتبع محرك المطالبات، ثم افحص الواجهة والبيانات الفعلية، وبعد ذلك أنشئ التقرير الكامل.